

Project 2: CIFAR-10 图像分类与 Batch Normalization 分析

姓名: 李宇喆

学号: 23307110173

代码仓库: <https://github.com/toparrc/FDU-26-nn-dl-Project-2>

本报告记录了 Project 2 中两个部分的实现与实验结果。第一部分是在 CIFAR-10 数据集上训练并优化卷积神经网络，重点比较网络深度、正则化强度、激活函数以及残差连接等因素对分类性能的影响。第二部分围绕 Batch Normalization 展开，使用 VGG-A 作为基础网络，对比加入 BN 前后的训练表现，并进一步从 loss landscape 的角度观察 BN 对优化过程的影响。

整体上，本项目的实验重点不是单纯堆叠更复杂的模型，而是在相对清晰的实验设置下理解不同结构和训练策略的作用。Task 1 中的基线是我们自己实现的 CIFAR-10 CNN；Task 2 中的基线才是 VGG-A without BN，二者在报告中分开讨论。

应任务要求，我们保存了模型的权重，仓库链接如下：

通过网盘分享的文件: PJ2-checkpoint.zip

链接: <https://pan.baidu.com/s/1aSYma4uqEmXt0JxLKdvV0w> 提取码: vze2

1. 实验设置

本项目使用 CIFAR-10 数据集。CIFAR-10 包含 60,000 张 32×32 彩色图像，共 10 个类别，每类 6,000 张。其中 50,000 张用于训练，10,000 张用于测试。我们从训练集中划分出 5,000 张作为验证集，用于模型选择和实验对比；测试集只用于最终评估。

训练时，输入图像经过常规张量化，归一化处理以及随机翻转、裁剪等数据增强方式。为了使不同实验之间具有可比性，除被考察因素外，训练配置尽量保持一致。

设置	Task 1 CNN	Task 2 VGG-A / VGG-A-BN
Dataset	CIFAR-10	CIFAR-10
Train / Val / Test	45,000/5,000 / 10,000	45,000/5,000/10,000
Batch size	128	128
Epochs	50	50
Optimizer	AdamW	Adam

设置	Task 1 CNN	Task 2 VGG-A / VGG-A-BN
Initial learning rate	10^{-3}	10^{-3} for BN comparison
Loss function	Cross Entropy	Cross Entropy
Scheduler	CosineAnnealingLR	None
Random seed	42	2020

Task 1 中我们主要比较验证集准确率、测试集准确率、训练时间和模型参数量。Task 2 中除了验证和测试性能外，还保存了不同 learning rate 下的 batch-level training loss，用于构造 loss landscape 的 min/max envelope。

2. 实现内容概述

本项目的代码分为两个相对独立的部分：`codes/CIFAR10_CNN/` 对应 Task 1，`codes/VGG_BatchNorm/` 对应 Task 2。下面先按文件说明实现内容，再集中说明各模型类。

2.1 Task 1: CIFAR-10 CNN 实验代码

- `data.py`：构建 CIFAR-10 的 train / validation / test dataloader。训练集使用 `RandomCrop` 和 `RandomHorizontalFlip` 做数据增强，验证集和测试集只做归一化，并用固定随机种子从训练集中划分 validation subset。
- `model.py`：定义 Task 1 的 CNN 模型、激活函数选择和权重初始化函数。模型部分包括普通四层 CNN、可配置深度的 plain CNN、三层 CNN 以及 residual CNN。
- `train.py`：实现单个模型的完整训练流程，包括训练、验证、测试、early stopping、checkpoint 保存、history CSV 保存、validation 曲线绘制和测试集 confusion matrix 绘制。
- `run_task1_experiments.py`：组织 Task 1 的自动对比实验，支持 regularization、activation、depth 和 optimizer 几组实验，并为每组实验保存对比曲线和 summary。
- `plots.py`：封装绘图函数，包括 validation loss / accuracy 曲线、不同配置之间的 validation 对比图和 confusion matrix。
- `compare_histories.py`：用于读取不同实验的 history 文件并进行额外对比分析，辅助整理实验结果。

Task 1 的主要模型类定义在 `model.py` 中：

- `CIFAR10CNN`：最初实现的基础 CNN，包含四个卷积 stage，每个 stage 使用卷积、BatchNorm、ReLU、MaxPool 和 Dropout。
- `CIFAR10PlainCNN`：通用 plain CNN 基类，通过传入通道配置构造不同深度的网络。

- `CIFAR10CNN4` : 四个卷积 stage 的主要 baseline, 通道数为 (64, 128, 256, 512)。
- `CIFAR10CNN3` : 减少一个卷积 stage 的较浅模型, 通道数为 (64, 128, 256), 用于比较网络深度影响。
- `ResidualBlock` : 残差模块, 主分支为两层卷积, shortcut 在通道数变化时使用 1x1 卷积对齐维度。
- `CIFAR10ResidualCNN` : 由四个 `ResidualBlock` 组成的 residual CNN, 用于观察残差连接对训练和泛化的影响。

2.2 Task 2: VGG-A 与 Batch Normalization 实验代码

- `models/vgg.py` : 定义 VGG 系列模型, 包括标准 VGG-A、轻量版 VGG-A、Dropout 版本, 以及它们对应的 BatchNorm 版本。
- `data/loaders.py` : 构建 Task 2 使用的 CIFAR-10 dataloader, 提供训练、验证和测试数据。
- `training_framework.py` : 封装通用训练框架, 包括随机种子设置、训练和验证循环、测试评估、batch-level loss 记录、梯度范数记录, 以及 min/max curve 构造函数。
- `VGG_BN_Comparison.py` : 训练并比较 `VGG_A` 和 `VGG_A_BatchNorm`, 保存 history CSV、最佳模型、summary, 以及 validation loss / accuracy 对比图。
- `VGG_Loss_Landscape.py` : 在多个 learning rate 下分别训练 `VGG_A` 和 `VGG_A_BatchNorm`, 保存每个 batch 的 loss、梯度范数、梯度变化量和梯度平滑性指标, 并构造 loss / gradient 的 min-max envelope。
- `utils/nn.py` : 提供 VGG 系列模型使用的权重初始化函数。

Task 2 的模型类主要定义在 `models/vgg.py` 中:

- `VGG_A` : 不含 BatchNorm 的标准 VGG-A baseline, 包含五个卷积 stage 和三层全连接分类器。
- `VGG_A_BatchNorm` : 在 `VGG_A` 的每个卷积层之后加入 `BatchNorm2d`, 用于分析 BN 对收敛速度、稳定性和泛化性能的影响。
- `VGG_A_Light` : 轻量版 VGG-A, 使用更少的 stage 和更小的通道数, 便于快速实验。
- `VGG_A_Light_BatchNorm` : `VGG_A_Light` 的 BatchNorm 版本。
- `VGG_A_Dropout` : 在标准 VGG-A 的分类器部分加入 Dropout, 用于观察 Dropout 正则化效果。
- `VGG_A_Dropout_BatchNorm` : 同时包含 Dropout 和 BatchNorm 的 VGG-A 变体。
- `VGG_A_BN` : `VGG_A_BatchNorm` 的别名, 方便代码中简写调用。

实际实验中, `VGG_BN_Comparison.py` 和 `VGG_Loss_Landscape.py` 主要使用 `VGG_A` 与 `VGG_A_BatchNorm` 进行对比; 其余模型作为扩展实现保留在 `models/vgg.py` 中。

3. Task 1: CIFAR-10 CNN 分类

3.1 基础 CNN 与残差模型

我们首先训练基础 CNN 和带 residual connection 的 CNN。基础 CNN 的输入为 $3 \times 32 \times 32$ 的 CIFAR-10 彩色图像，主体由四个卷积 stage 组成。每个 stage 都包含两层 3×3 convolution、BatchNorm、激活函数、 2×2 max pooling 和 Dropout。经过四次池化后，空间分辨率从 32×32 下降到 2×2 ，最后接两层全连接分类器输出 10 类 logits。

模型权重初始化统一使用 Kaiming normal: Conv2d 和 Linear 的 weight 使用 `kaiming_normal_`，bias 初始化为 0；BatchNorm 的 scale 参数初始化为 1，shift 参数初始化为 0。基础 CNN 的结构如下：

模块	输出通道 / 维度	主要操作
Input	$3 \times 32 \times 32$	CIFAR-10 image
Stage 1	$64 \times 16 \times 16$	$2 \times (\text{Conv}3 \times 3\text{-BN-Act}), \text{MaxPool}, \text{Dropout}2d(p = 0.0)$
Stage 2	$128 \times 8 \times 8$	$2 \times (\text{Conv}3 \times 3\text{-BN-Act}), \text{MaxPool}, \text{Dropout}2d(p = 0.0)$
Stage 3	$256 \times 4 \times 4$	$2 \times (\text{Conv}3 \times 3\text{-BN-Act}), \text{MaxPool}, \text{Dropout}2d(p = 0.1)$
Stage 4	$512 \times 2 \times 2$	$2 \times (\text{Conv}3 \times 3\text{-BN-Act}), \text{MaxPool}, \text{Dropout}2d(p = 0.1)$
Classifier	$2048 \rightarrow 512 \rightarrow 10$	Flatten, Linear-Act, Dropout($p = 0.3$), Linear

Residual CNN 保留了相同的 4-stage 宏观结构，但把每个 stage 中的两层卷积改成 residual block。每个 block 的主分支是 Conv-BN-Act-Conv-BN，shortcut 分支在输入输出通道一致时使用 identity；当通道数改变时，使用 1×1 convolution 和 BatchNorm 对齐维度。随后将主分支和 shortcut 相加，再经过激活、池化和 Dropout。这样做的目的是让特征和梯度可以通过 shortcut 更直接地传播，缓解较深 CNN 训练时的信息损失。

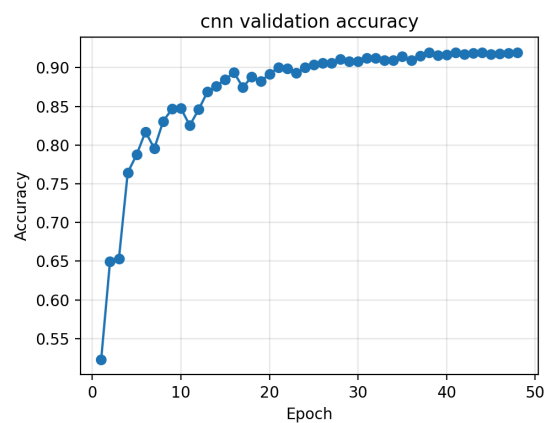
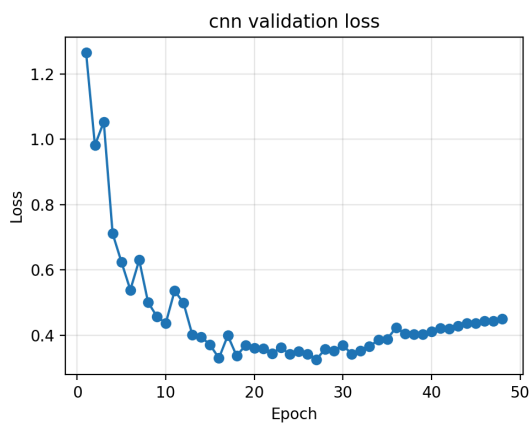
Residual CNN (本实验中测试准确率最高的模型配置)，其具体层设计如下：

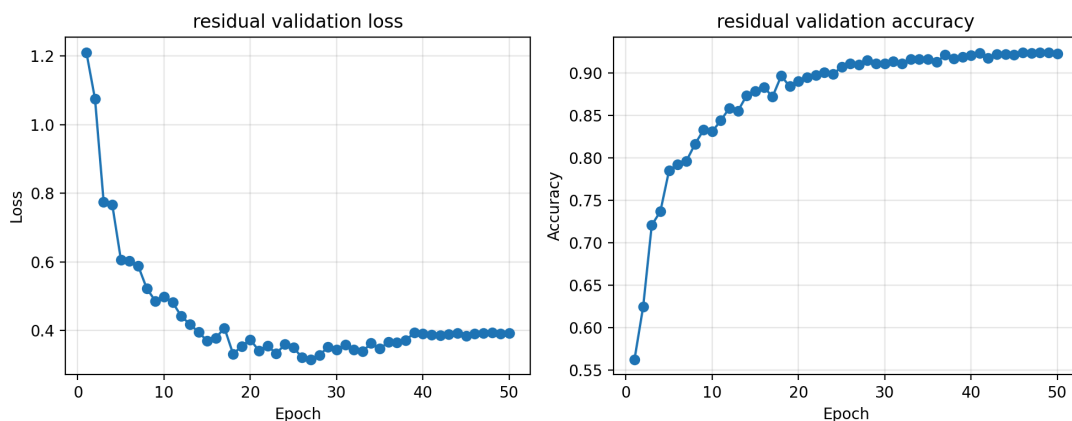
模块	输出通道 / 维度	主要操作
Input	$3 \times 32 \times 32$	CIFAR-10 image
Residual Block 1	$64 \times 16 \times 16$	Main: Conv3x3-BN-Act-Conv3x3-BN

模块	输出通道 / 维度	主要操作
		Shortcut: Conv1x1-BN Add-Act, MaxPool, Dropout2d($p = 0.0$)
Residual Block 2	$128 \times 8 \times 8$	Main: Conv3x3-BN-Act-Conv3x3-BN Shortcut: Conv1x1-BN Add-Act, MaxPool, Dropout2d($p = 0.0$)
Residual Block 3	$256 \times 4 \times 4$	Main: Conv3x3-BN-Act-Conv3x3-BN Shortcut: Conv1x1-BN Add-Act, MaxPool, Dropout2d($p = 0.1$)
Residual Block 4	$512 \times 2 \times 2$	Main: Conv3x3-BN-Act-Conv3x3-BN Shortcut: Conv1x1-BN Add-Act, MaxPool, Dropout2d($p = 0.1$)
Classifier	$2048 \rightarrow 512 \rightarrow 10$	Flatten, Linear-Act, Dropout($p = 0.3$), Linear

模型训练和测试结果如下：

模型	参数量	Best Epoch	Best Val Acc	Test Acc	Test Error	Test Loss
CNN	5,743,434	38	91.98%	90.79%	9.21%	0.4585
Residual CNN	5,915,658	49	92.40%	91.76%	8.24%	0.4195



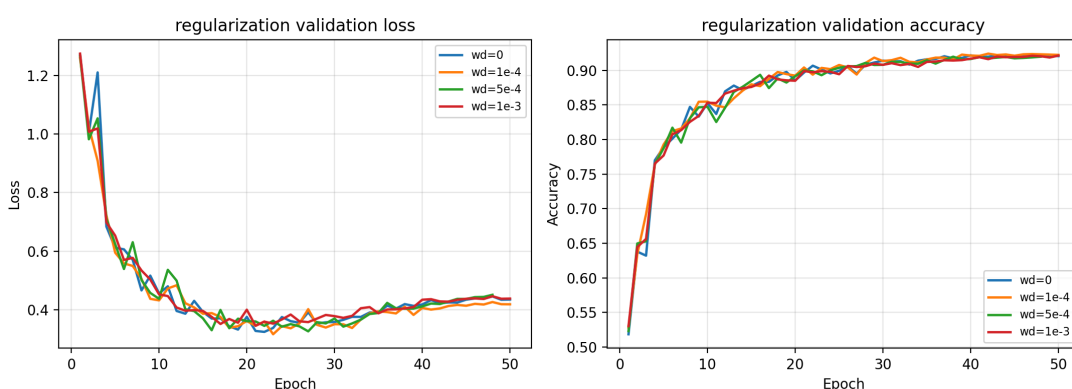


从结果看，Residual CNN 的参数量只比基础 CNN 略高，但测试准确率从 90.79% 提升到 91.76%。这个提升不算巨大，但比较稳定，说明在当前深度下 residual connection 已经可以改善信息传递和优化过程。基础 CNN 在第 38 个 epoch 达到最佳验证准确率，而 residual 模型的最佳 epoch 出现在第 49 个 epoch，说明它在训练后期仍然保持改进趋势。如果进一步延长训练轮数，residual 模型仍可能获得额外提升；这也符合残差连接在更深网络中通常更有利于梯度传播和优化的经验。

3.2 正则化强度比较

为了观察 weight decay 对泛化能力的影响，我们固定基础模型为 `cnn4`，激活函数为 ReLU，优化器为 AdamW，初始学习率为 10^{-3} ，只改变 weight decay。

Weight Decay	Best Epoch	Best Val Acc	Test Acc	Test Loss
0	49	92.20%	91.47%	0.4793
10^{-4}	42	92.42%	91.30%	0.4420
5×10^{-4}	38	91.98%	90.79%	0.4585
10^{-3}	50	92.14%	91.50%	0.4572

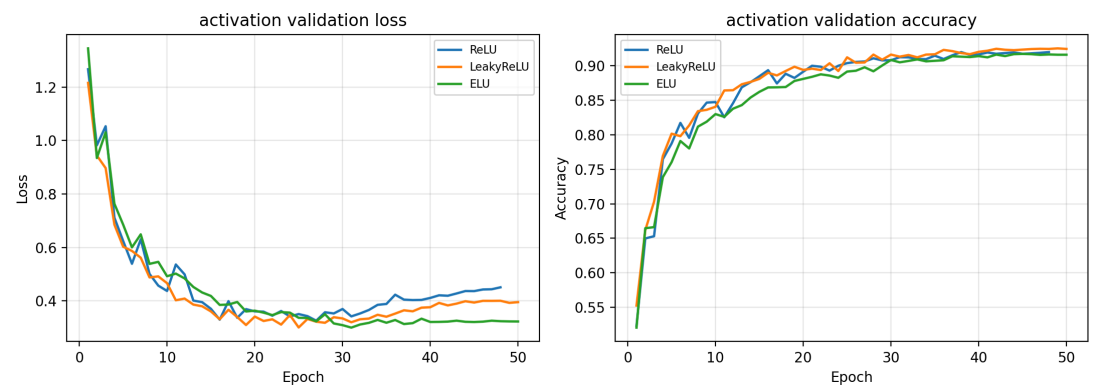


这组实验里，weight decay 的影响比较细微。 10^{-4} 取得最高验证准确率 92.42%，但测试准确率并没有超过 10^{-3} 和不加 weight decay 的设置。 10^{-3} 的测试准确率最高，为 91.50%，但验证准确率不是最高。这说明当前模型在 CIFAR-10 上对 weight decay 的敏感性存在一定随机波动，较小或中等强度的正则化都有合理表现；过早根据单一验证点判断最优参数并不稳妥。

3.3 激活函数比较

接着固定基础模型为 `cnn4`，优化器为 AdamW，初始学习率为 10^{-3} ，weight decay 为 5×10^{-4} ，比较 ReLU、LeakyReLU 和 ELU。

Activation	Best Epoch	Best Val Acc	Test Acc	Test Loss
ReLU	38	91.98%	90.79%	0.4585
LeakyReLU	49	92.52%	91.41%	0.4630
ELU	45	91.72%	90.82%	0.3523

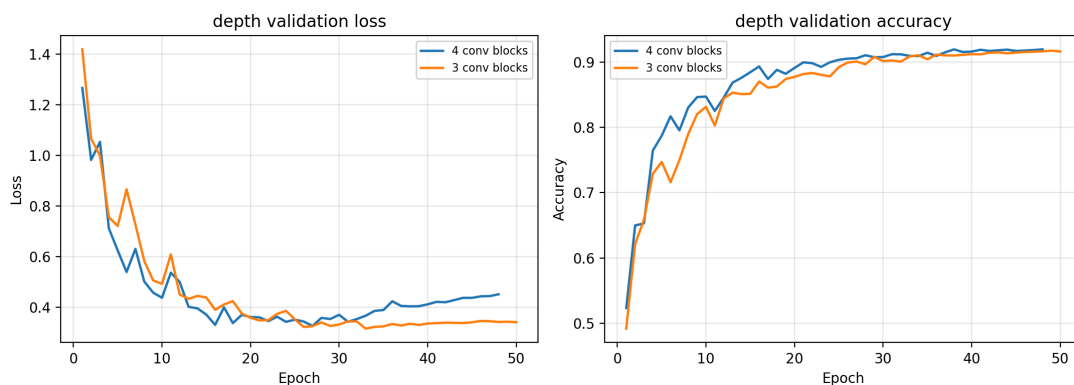


LeakyReLU 在验证准确率和测试准确率上都优于 ReLU，说明保留负半轴的小斜率对当前网络有一定帮助。ELU 的 test loss 最低，但 test accuracy 并没有明显提高，这说明它输出的概率分布可能更平滑或更保守，但分类决策本身没有显著优势。综合准确率表现，LeakyReLU 是这组实验中更合适的选择。

3.4 网络深度比较

我们还比较了四个卷积 stage 的 `cnn4` 和三个卷积 stage 的 `cnn3`。二者使用相同的训练配置：ReLU 激活函数、AdamW 优化器、初始学习率 10^{-3} 、weight decay 5×10^{-4} ，主要区别是卷积深度和参数量。

Model	参数量	Best Epoch	Best Val Acc	Test Acc	Avg Epoch Time
4 conv blocks	5,743,434	38	91.98%	90.79%	13.90 s
3 conv blocks	3,249,994	49	91.78%	90.66%	12.10 s

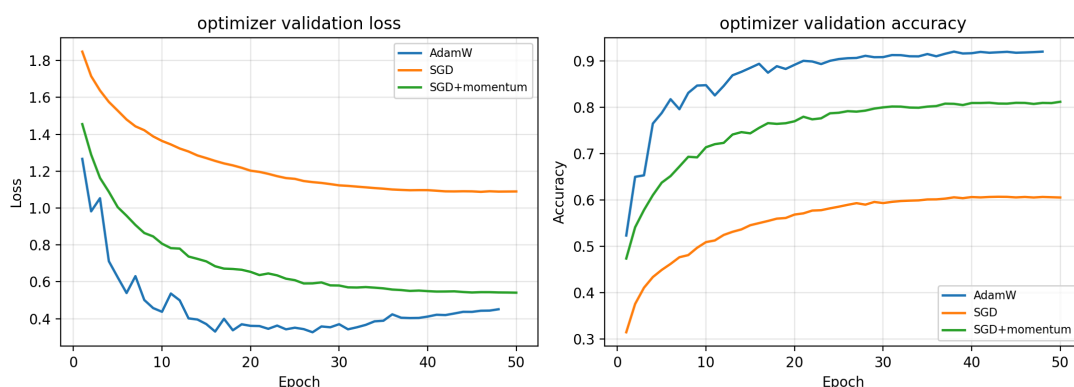


更深的 4-stage CNN 只带来了很小的准确率提升，但参数量从约 $3.25M$ 增加到 $5.74M$ 。这说明在当前训练配置下，额外 stage 的收益有限。较浅模型速度更快、参数更少，在性能接近时也有实际价值；如果追求最高准确率，可以使用更深模型或 residual 模型，但如果考虑效率，4-stage CNN 是一个更轻量的选择。

3.5 优化器比较

为了比较优化器对训练动态和最终性能的影响，我们固定模型为 `cnn4`，激活函数为 ReLU，weight decay 为 5×10^{-4} ，初始学习率为 10^{-3} ，分别使用 AdamW、SGD 和 SGD with momentum 训练。三组实验均使用 batch size 128、最多 50 个 epoch 和 CosineAnnealingLR scheduler。

Optimizer	LR	Weight Decay	Momentum	Best Epoch	Best Val Acc	Test Acc	Test Loss	Avg Epoch Time
AdamW	10^{-3}	5×10^{-4}	0	38	91.98%	90.79%	0.4585	9.89 s
SGD	10^{-3}	5×10^{-4}	0	43	60.68%	60.70%	1.1007	9.44 s
SGD with momentum	10^{-3}	5×10^{-4}	0.9	50	81.16%	80.42%	0.5631	9.63 s

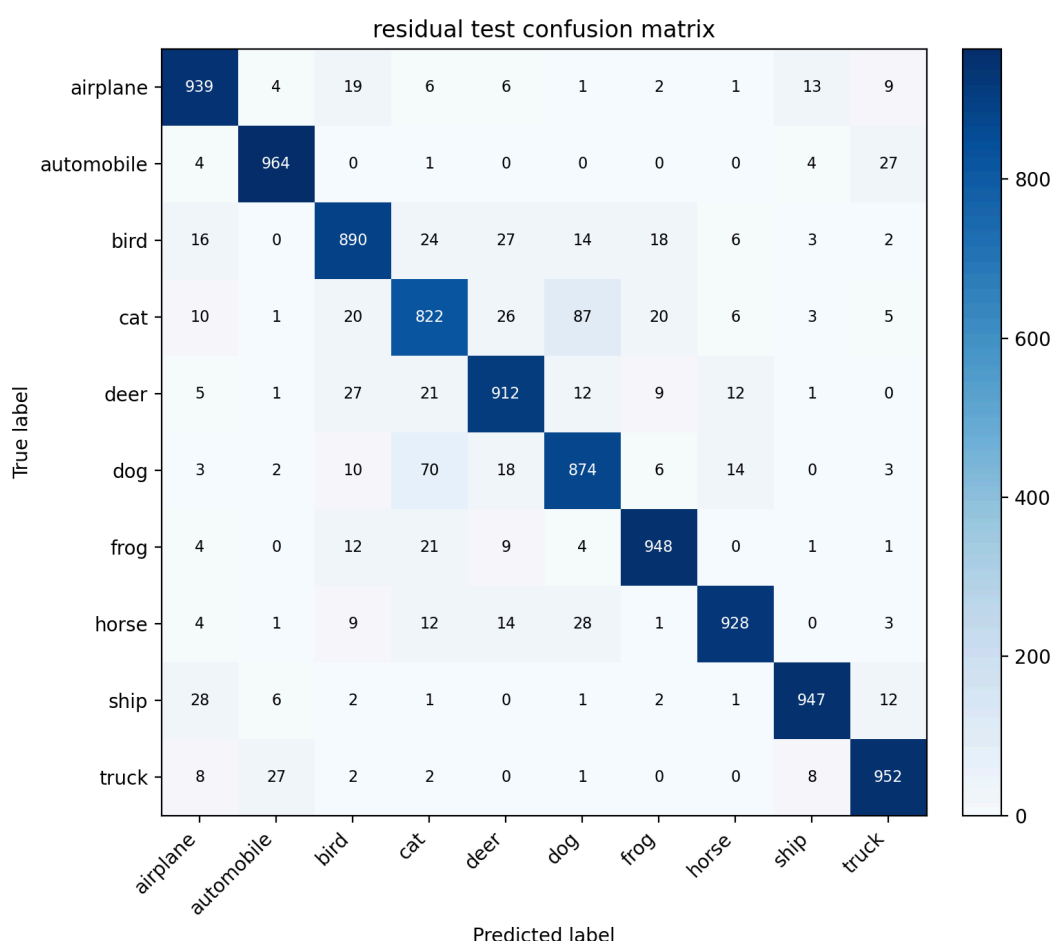


在相同学习率下，AdamW 的表现明显最好，验证准确率和测试准确率都超过 90%。CIFAR-10 图像分类对优化器并不算特别简单：模型包含多层卷积、BatchNorm、Dropout 和数据增强，不同层、不同参数的梯度尺度差异较大，mini-batch 梯度也带有较强噪声。AdamW 同时利用一阶矩估计累积稳定的下降方向，并利用二阶矩估计为每个参数自适应调整有效步长，因此在这种非凸、梯度尺度不均匀的 CNN 训练中更容易快速

进入较好的优化区域。另一方面，AdamW 将 weight decay 与梯度更新解耦，正则化强度不会直接被自适应学习率扭曲，这也有利于在保持泛化能力的同时稳定收敛。

相比之下，普通 SGD 在所有参数上使用同一个全局学习率，面对不同层梯度尺度差异时更依赖精细的学习率调节，因此在本实验的 10^{-3} 设置下收敛明显偏慢，50 个 epoch 内只达到约 60.70% 的测试准确率。加入 momentum 后，优化器能够累积连续 batch 中一致的梯度方向，减弱随机梯度噪声和局部震荡，所以测试准确率提升到 80.42%。但 momentum 仍然缺少 AdamW 的逐参数自适应步长，在当前超参数下仍落后于 AdamW。这个结果并不意味着 SGD 本身一定较差，而是说明在本实验统一使用 10^{-3} 初始学习率和 CosineAnnealingLR 时，AdamW 的自适应更新机制更适合当前 CNN 的训练；SGD 若要达到更强表现，通常需要单独调大学习率、调整 scheduler 或延长训练轮数。

3.6 混淆矩阵与错误类型



从混淆矩阵可以看出，模型在一些视觉相似类别上更容易混淆。例如 cat / dog / deer / horse 等动物类别之间的错误通常多于 airplane / ship / automobile / truck 这类轮廓更明显的类别。这符合 CIFAR-10 的直观难度：低分辨率图像中动物姿态、背景和纹理变化较大，而交通工具类别往往具有更稳定的全局形状。

3.7 训练后期 loss 与 accuracy 的分叉现象

在 Task 1 的若干实验中还可以观察到一个有意思的现象：四层 CNN 在训练后期，尤其是大约第 30 到 50 个 epoch 之间，validation loss 有回升趋势，但 validation accuracy 仍然保持上升或基本维持在较高水平。例如 ReLU 版本在第 30 个 epoch 的 validation loss 约为 0.3699、validation accuracy 为 90.80%，到第 48 个 epoch 时 validation loss 上升到约 0.4510，但 validation accuracy 仍达到 91.98%。LeakyReLU 版本也有类似现象，后期 accuracy 继续小幅提升，但 loss 没有同步下降。相比之下，三层 CNN 的 validation loss 在后期更加平稳，ELU 作为激活函数时这种 loss 回升也不明显。

我们经过猜想和一定探究，认为这个现象的核心原因是 cross-entropy loss 和 accuracy 衡量的对象不同。Accuracy 只关心预测类别是否等于真实标签，也就是 logits 中最大值的位置是否正确；而 cross-entropy 不仅关心分类是否正确，还关心模型给真实类别分配了多大的概率。对于一个已经分对的样本，只要真实类别仍然是最大概率类别，accuracy 就不会变化；但对于分错的样本，如果模型对错误类别给出非常高的置信度，cross-entropy 会受到很大惩罚。因此训练后期可能出现这样的情况：模型把一部分原本边界附近的样本分对了，使 accuracy 上升；同时对少量仍然分错的样本变得更加自信，使这些错误样本贡献更大的 loss，最终造成 validation loss 回升。

四层 CNN 更容易出现这种现象，可能与模型容量更强有关。更深的网络在训练后期可以继续降低训练 loss，并把 logits 推得更“尖锐”，也就是预测分布更加接近 one-hot。这对容易样本有利，但对验证集中存在噪声、歧义或者视觉上相似的样本来说，过高置信度会让错误预测的代价变大。三层 CNN 容量较小，后期 logits 的极化程度可能弱一些，因此 validation loss 没有明显回升。激活函数也会影响这个趋势：ReLU 和 LeakyReLU 的正半轴不饱和，较容易产生幅度更大的激活和 logits；而 ELU 在负半轴具有饱和性质，输出分布相对平滑，可能在一定程度上抑制过度自信，所以它的 validation loss 后期更加稳定。

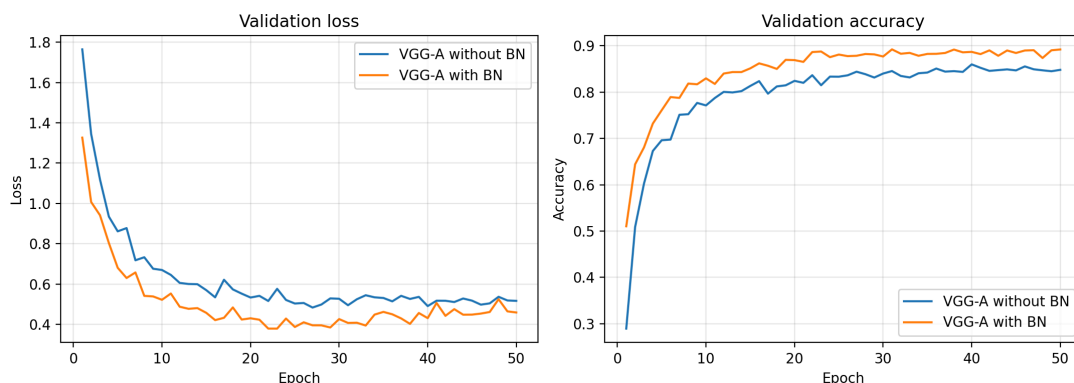
因此，这一现象并不直接说明模型性能退化，而是说明后期模型的概率校准变差：分类边界可能仍在改善，但预测置信度变得更激进。若希望同时降低 validation loss 并保持 accuracy，可以考虑更强的数据增强、label smoothing、temperature scaling、early stopping，或适当增大 dropout / weight decay 来缓解过度自信。

4. Task 2: VGG-A with and without Batch Normalization

4.1 BN 对训练性能的影响

在这一部分中，我们使用课程提供的 VGG-A 结构作为 baseline，然后在每个 convolution layer 后加入 BatchNorm2d，比较两者在 CIFAR-10 上的训练结果。

模型	Best Epoch	Best Val Acc	Final Val Acc	Test Acc	Test Loss
VGG-A without BN	40	86.00%	84.82%	84.81%	0.5128
VGG-A with BN	31	89.24%	89.22%	88.33%	0.4467



加入 BN 后，VGG-A 的最佳验证准确率从 86.00% 提升到 89.24%，测试准确率从 84.81% 提升到 88.33%。同时，BN 模型在第 31 个 epoch 就达到最佳验证结果，而不含 BN 的模型在第 40 个 epoch 才达到最佳结果。这说明 BN 不只是提升了最终性能，也让训练过程更快进入较好的区域。

从训练曲线看，BN 模型的训练准确率最终达到约 97.53%，明显高于无 BN 模型的 91.25%。这并不简单等价于过拟合，因为它的验证和测试准确率也同步更高。更合理的解释是，BN 改善了深层卷积网络的优化条件，使模型更容易拟合训练数据，同时保持更好的泛化性能。

具体来说，BatchNorm 在每个 mini-batch 内对中间特征做标准化，使特征在进入下一层之前具有更稳定的均值和方差。这样可以减小不同 batch 之间激活分布的剧烈波动，也能缓解前面层参数更新后对后面层输入尺度造成的影响。标准化之后，BN 并不是简单地把所有特征固定成同一种分布，而是引入可学习的缩放参数 γ 和平移参数 β ，让网络自己决定每个通道需要保留多大的尺度和偏移。因此 BN 一方面提供了统一、稳定的特征尺度，另一方面又没有牺牲模型表达能力。

从优化角度看，BN 相当于对网络做了一种重参数化。未加 BN 时，某一层权重尺度的变化会直接放大或缩小后续层的输入，使 loss surface 对参数尺度更敏感；加入 BN 后，中间激活会被重新归一化，优化器看到的有效参数空间更加稳定。这样梯度更新不容易因为某些层的激活尺度过大或过小而失衡，较大的学习率也更容易工作。因此，BN 模型不仅收敛更快，而且最终能达到更高的训练、验证和测试准确率。

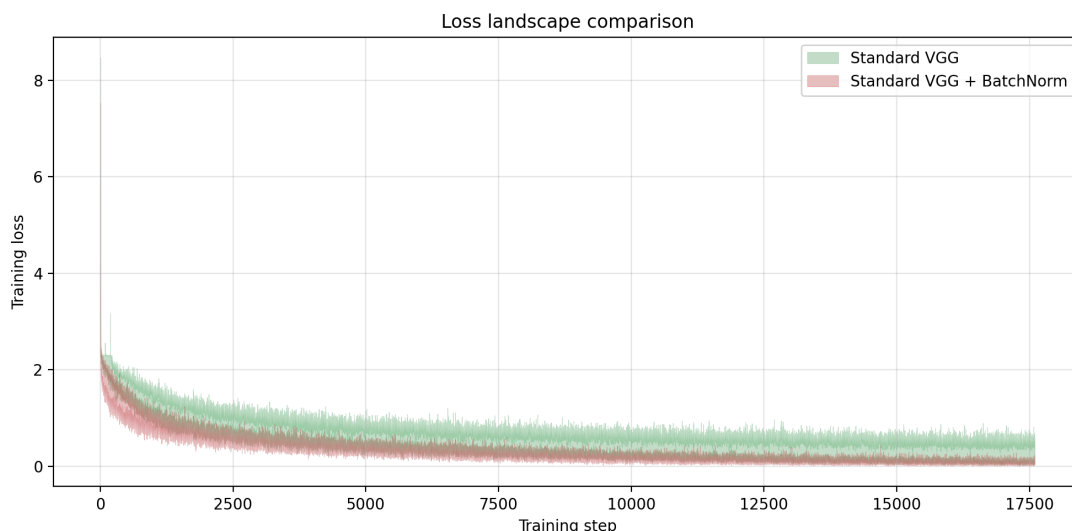
4.2 Loss Landscape

为了进一步观察 BN 对优化过程的影响，我们按照任务要求选取多个 learning rate:

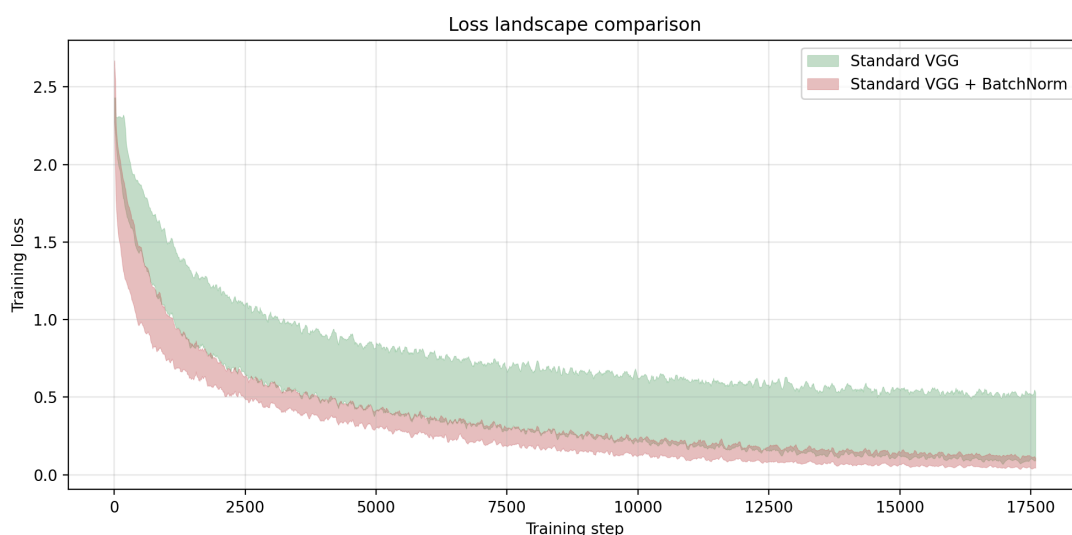
$$[10^{-3}, 2 \times 10^{-3}, 10^{-4}, 5 \times 10^{-4}]$$

对 VGG-A 和 VGG-A-BN 分别训练，并保存每个 batch 的 training loss。对于同一个训练 step，我们在不同 learning rate 的运行结果中取 loss 的最小值和最大值，构造 `min_curve` 和 `max_curve`，再用 `fill_between` 画出 envelope。这个区域越窄，说明不同 step size 下 loss 变化越稳定；区域越宽，则说明优化过程对 step size 更敏感。

首先直接绘制未平滑的 batch-level loss envelope:



未平滑图保留了每个 mini-batch 的原始波动，但由于训练 step 很密集，batch-level loss 又受到数据采样噪声影响，曲线局部抖动较强，部分 envelope 区域会被高频波动遮住，不利于观察整体趋势。因此，在最终分析图中我加入了移动窗口平均，并适当下采样绘图点，使图像更突出不同 learning rate 下 loss 轨迹的整体包络。



从 loss landscape 图可以看到，加入 BN 后的 envelope 整体更窄、更平滑，尤其是在训练早期，无 BN 模型对 learning rate 的变化更加敏感，loss 曲线之间的差异更明显。BN 模型的 loss 下降更稳定，这与 BN 能够 smooth optimization landscape 的解释是一致的。

这个实验不直接证明 BN 的全部机制，但它提供了一个可视化角度：BN 重新参数化了网络，使得相邻训练轨迹下的 loss 变化更加平缓，因此一阶优化方法的局部近似更可靠，训练也更容易稳定推进。

5. 总结与讨论

本项目中，我们首先在 CIFAR-10 上实现并训练了自定义 CNN。基础 CNN 已经可以达到约 90.79% 的测试准确率；加入 residual connection 后，测试准确率提升到 91.76%，说明残差连接对当前深度的卷积网络仍然有积极作用。正则化、激活函数和网

络深度的对比实验显示，模型表现并不只由参数量决定：LeakyReLU 相比 ReLU 有更好的准确率表现；较深模型相比较浅模型提升有限；weight decay 的最佳取值也不是越大越好，而是在泛化和拟合能力之间取得平衡。

在 BN 部分，VGG-A-BN 相比 VGG-A without BN 有明显提升。加入 BN 后，最佳验证准确率提升约 3.24 个百分点，测试准确率提升约 3.52 个百分点，并且更早达到最佳验证性能。loss landscape 实验进一步显示，BN 模型在不同 learning rate 下的 loss envelope 更窄，说明其优化过程对 step size 的敏感性更低，训练轨迹更加平滑稳定。

当前实验仍有一些限制。Task 1 的 optimizer 对比使用了统一的初始学习率和 scheduler，因此它更能说明当前训练配置下不同优化器的相对表现，而不是每个优化器在充分调参后的最优性能。Task 2 的 loss landscape 也主要是通过不同 learning rate 下的 batch-level loss envelope 来观察优化稳定性，并不等价于对完整高维 loss surface 的严格刻画。

总体而言，本项目完成了 CIFAR-10 分类网络的设计与优化实验，也验证了 Batch Normalization 对 VGG-A 训练性能和 loss landscape 稳定性的改善作用。

```
In [2]: !jupyter nbconvert --to html --embed-images report.ipynb --output report.html
```

```
[NbConvertApp] Converting notebook report.ipynb to html
/home/toparrc1/anaconda3/envs/llm-26-gpu/lib/python3.12/site-packages/nbformat/__init__.py:96: MissingIDFieldWarning: Cell is missing an id field, this will become a hard error in future nbformat versions. You may want to use `normalize()` on your notebooks before validations (available since nbformat 5.1.4). Previous versions of nbformat are fixing this issue transparently, and will stop doing so in the future.
  validate(nb)
[NbConvertApp] Writing 2109473 bytes to report.html
```